NAME : Madhu Sudhan Reddy B

REG NO : 192312242

COURSE CODE : ITA0613

COURSE NAME : MACHINE LEARNING

Candidate-Elimination Algorithm: Version Space of Consistent Hypotheses

Implementation and demonstration of the Candidate-Elimination algorithm using training examples stored in CSV files, for five different data sets. Each experiment contains the Aim, Algorithm, Python Code, Output and Result.

Question 1: EnjoySport Concept Learning

Problem Statement

The EnjoySport training examples are stored in the CSV file enjoysport.csv. Use the Candidate-Elimination algorithm to find the set of all hypotheses consistent with the data.

Training data (enjoysport.csv):

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Aim

To read training examples from a CSV file and implement the Candidate-Elimination algorithm in Python to output the version space (S and G boundaries) consistent with the EnjoySport data.

Algorithm

1. Read the training examples from the CSV file (last column is the class label).
2. Initialise G to the most general hypothesis {<?, ?, ..., ?>} and S to the most specific hypothesis {<0, 0, ..., 0>}.
3. For each positive example: remove from G every hypothesis that does not cover it.

4. For each hypothesis s in S that does not cover the positive example, replace it by its minimal generalization.
5. For each negative example: remove from S every hypothesis that covers it.
6. For each hypothesis g in G that covers the negative example, replace it by its minimal specializations that are still more general than some member of S, and keep only the most general ones.
7. Print S and G after every example and output the final S and G, which describe the version space.

Python Code

```
import csv

def load(fn):
    with open(fn) as f:
        r = list(csv.reader(f))
    return r[0], r[1:]

def covers(h, x):
    return all(a == "?" or a == b
               for a, b in zip(h, x))

def more_gen(h1, h2):
    return all(a == "?" or a == b or b == "0"
               for a, b in zip(h1, h2))
```

```
def fmt(H):
    return " ".join("<" + ", ".join(h) + ">"
                    for h in H)

def uniq(hs):
    out = []
    for h in hs:
        if h not in out:
            out.append(h)
    return out

def candidate_elimination(rows):
    n = len(rows[0]) - 1
    dom = [sorted({r[i] for r in rows})
           for i in range(n)]
    S, G = [["0"] * n], [["?"] * n]
    print("S0 =", fmt(S))
    print("G0 =", fmt(G))
    for t, r in enumerate(rows, 1):
        x, label = r[:-1], r[-1]
        if label == "Yes":
            G = [g for g in G if covers(g, x)]
```

```
            S = [[xv if sv == "0" else
                  (sv if sv == xv else "?")
                  for sv, xv in zip(s, x)]
                 for s in S]
        else:
            S = [s for s in S if not covers(s, x)]
            new = []
            for g in G:
                if not covers(g, x):
                    new.append(g)
                    continue
                for i in range(n):
                    if g[i] != "?":
                        continue
                    for v in dom[i]:
                        if v != x[i]:
                            h = g[:]
                            h[i] = v
                            if any(more_gen(h, s)
                                   for s in S):
                                new.append(h)
            G = [g for g in uniq(new)
                 if not any(g != k and more_gen(k, g)
```

```
                        for k in new)]
        print(f"\nExample {t}: {x} -> {label}")
        print("S =", fmt(S))
        print("G =", fmt(G))
    return S, G

header, rows = load("enjoysport.csv")
print("Attributes:", header[:-1])
S, G = candidate_elimination(rows)
print("\nFinal Specific Boundary S:", fmt(S))
print("Final General Boundary G :", fmt(G))
```

Output

```
Attributes: ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
S0 = <0, 0, 0, 0, 0, 0>
G0 = <?, ?, ?, ?, ?, ?>

Example 1: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'] -> Yes
S = <Sunny, Warm, Normal, Strong, Warm, Same>
G = <?, ?, ?, ?, ?, ?>

Example 2: ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same'] -> Yes
S = <Sunny, Warm, ?, Strong, Warm, Same>
```

G = <?, ?, ?, ?, ?, ?>

Example 3: ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change'] -> No
S = <Sunny, Warm, ?, Strong, Warm, Same>
G = <Sunny, ?, ?, ?, ?, ?> <?, Warm, ?, ?, ?, ?> <?, ?, ?, ?, ?, Same>

Example 4: ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change'] -> Yes
S = <Sunny, Warm, ?, Strong, ?, ?>
G = <Sunny, ?, ?, ?, ?, ?> <?, Warm, ?, ?, ?, ?>

Final Specific Boundary S: <Sunny, Warm, ?, Strong, ?, ?>
Final General Boundary G : <Sunny, ?, ?, ?, ?, ?> <?, Warm, ?, ?, ?, ?>

Result

The Candidate-Elimination algorithm gave S = <Sunny, Warm, ?, Strong, ?, ?> and G = {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>}. All hypotheses between S and G form the version space consistent with the training examples.

Question 2: Weather Data (Play Tennis)

Problem Statement

The weather observations are stored in weather.csv. Use Candidate-Elimination to find all hypotheses consistent with the days on which tennis was played.

Training data (weather.csv):

Outlook	Temp	Humidity	Wind	PlayTennis
Sunny	Hot	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Rain	Cool	High	Weak	No
Overcast	Mild	High	Strong	No

Aim

To apply the Candidate-Elimination algorithm on the weather data stored in a CSV file and obtain the specific and general boundaries of the version space.

Algorithm

1. Read the training examples from the CSV file (last column is the class label).
2. Initialise G to the most general hypothesis {<?, ?, ..., ?>} and S to the most specific hypothesis {<0, 0, ..., 0>}.
3. For each positive example: remove from G every hypothesis that does not cover it.

4. For each hypothesis s in S that does not cover the positive example, replace it by its minimal generalization.
5. For each negative example: remove from S every hypothesis that covers it.
6. For each hypothesis g in G that covers the negative example, replace it by its minimal specializations that are still more general than some member of S, and keep only the most general ones.
7. Print S and G after every example and output the final S and G, which describe the version space.

Python Code

```
import csv

def load(fn):
    with open(fn) as f:
        r = list(csv.reader(f))
    return r[0], r[1:]

def covers(h, x):
    return all(a == "?" or a == b
               for a, b in zip(h, x))

def more_gen(h1, h2):
    return all(a == "?" or a == b or b == "0"
               for a, b in zip(h1, h2))
```

```
def fmt(H):
    return " ".join("<" + ", ".join(h) + ">"
                    for h in H)

def uniq(hs):
    out = []
    for h in hs:
        if h not in out:
            out.append(h)
    return out

def candidate_elimination(rows):
    n = len(rows[0]) - 1
    dom = [sorted({r[i] for r in rows})
           for i in range(n)]
    S, G = [["0"] * n], [["?"] * n]
    print("S0 =", fmt(S))
    print("G0 =", fmt(G))
    for t, r in enumerate(rows, 1):
        x, label = r[:-1], r[-1]
        if label == "Yes":
            G = [g for g in G if covers(g, x)]
```

```
            S = [[xv if sv == "0" else
                  (sv if sv == xv else "?")
                  for sv, xv in zip(s, x)]
                 for s in S]
        else:
            S = [s for s in S if not covers(s, x)]
            new = []
            for g in G:
                if not covers(g, x):
                    new.append(g)
                    continue
                for i in range(n):
                    if g[i] != "?":
                        continue
                    for v in dom[i]:
                        if v != x[i]:
                            h = g[:]
                            h[i] = v
                            if any(more_gen(h, s)
                                   for s in S):
                                new.append(h)
            G = [g for g in uniq(new)
                 if not any(g != k and more_gen(k, g)
```

```
                        for k in new)]
        print(f"\nExample {t}: {x} -> {label}")
        print("S =", fmt(S))
        print("G =", fmt(G))
    return S, G

header, rows = load("weather.csv")
print("Attributes:", header[:-1])
S, G = candidate_elimination(rows)
print("\nFinal Specific Boundary S:", fmt(S))
print("Final General Boundary G :", fmt(G))
```

Output

```
Attributes: ['Outlook', 'Temp', 'Humidity', 'Wind']
S0 = <0, 0, 0, 0>
G0 = <?, ?, ?, ?>

Example 1: ['Sunny', 'Hot', 'Normal', 'Weak'] -> Yes
S = <Sunny, Hot, Normal, Weak>
G = <?, ?, ?, ?>

Example 2: ['Sunny', 'Mild', 'Normal', 'Strong'] -> Yes
S = <Sunny, ?, Normal, ?>
```

G = <?, ?, ?, ?>

Example 3: ['Rain', 'Cool', 'High', 'Weak'] -> No
S = <Sunny, ?, Normal, ?>
G = <Sunny, ?, ?, ?> <?, ?, Normal, ?>

Example 4: ['Overcast', 'Mild', 'High', 'Strong'] -> No
S = <Sunny, ?, Normal, ?>
G = <Sunny, ?, ?, ?> <?, ?, Normal, ?>

Final Specific Boundary S: <Sunny, ?, Normal, ?>
Final General Boundary G : <Sunny, ?, ?, ?> <?, ?, Normal, ?>

Result

The final boundaries are S = <Sunny, ?, Normal, ?> and G = {<Sunny, ?, ?, ?>, <?, ?, Normal, ?>}. Tennis is played when the outlook is Sunny or the humidity is Normal, and at the specific end when both hold.

Question 3: Buys Computer Prediction

Problem Statement

Customer records are stored in computer.csv. Use Candidate-Elimination to find all hypotheses consistent with the customers who buy a computer.

Training data (computer.csv):

Age	Income	Student	Credit	BuysComputer
Youth	Medium	Yes	Fair	Yes
Youth	Low	Yes	Excellent	Yes
Senior	High	No	Fair	No
Youth	High	No	Fair	No
Middle	Medium	No	Excellent	No

Aim

To implement the Candidate-Elimination algorithm on the customer data stored in a CSV file and output the set of hypotheses consistent with the buying behaviour.

Algorithm

1. Read the training examples from the CSV file (last column is the class label).
2. Initialise G to the most general hypothesis {<?, ?, ..., ?>} and S to the most specific hypothesis {<0, 0, ..., 0>}.

3. For each positive example: remove from G every hypothesis that does not cover it.
4. For each hypothesis s in S that does not cover the positive example, replace it by its minimal generalization.
5. For each negative example: remove from S every hypothesis that covers it.
6. For each hypothesis g in G that covers the negative example, replace it by its minimal specializations that are still more general than some member of S, and keep only the most general ones.
7. Print S and G after every example and output the final S and G, which describe the version space.

Python Code

```
import csv

def load(fn):
    with open(fn) as f:
        r = list(csv.reader(f))
    return r[0], r[1:]

def covers(h, x):
    return all(a == "?" or a == b
               for a, b in zip(h, x))

def more_gen(h1, h2):
    return all(a == "?" or a == b or b == "0"
```

```
            for a, b in zip(h1, h2))

def fmt(H):
    return " ".join("<" + ", ".join(h) + ">"
                    for h in H)

def uniq(hs):
    out = []
    for h in hs:
        if h not in out:
            out.append(h)
    return out

def candidate_elimination(rows):
    n = len(rows[0]) - 1
    dom = [sorted({r[i] for r in rows})
           for i in range(n)]
    S, G = [["0"] * n], [["?"] * n]
    print("S0 =", fmt(S))
    print("G0 =", fmt(G))
    for t, r in enumerate(rows, 1):
        x, label = r[:-1], r[-1]
        if label == "Yes":
```

```
            G = [g for g in G if covers(g, x)]
            S = [[xv if sv == "0" else
                  (sv if sv == xv else "?")
                  for sv, xv in zip(s, x)]
                 for s in S]
        else:
            S = [s for s in S if not covers(s, x)]
            new = []
            for g in G:
                if not covers(g, x):
                    new.append(g)
                    continue
                for i in range(n):
                    if g[i] != "?":
                        continue
                    for v in dom[i]:
                        if v != x[i]:
                            h = g[:]
                            h[i] = v
                            if any(more_gen(h, s)
                                   for s in S):
                                new.append(h)
            G = [g for g in uniq(new)
```

```
            if not any(g != k and more_gen(k, g)
                       for k in new)]
        print(f"\nExample {t}: {x} -> {label}")
        print("S =", fmt(S))
        print("G =", fmt(G))
    return S, G

header, rows = load("computer.csv")
print("Attributes:", header[:-1])
S, G = candidate_elimination(rows)
print("\nFinal Specific Boundary S:", fmt(S))
print("Final General Boundary G :", fmt(G))
```

Output

```
Attributes: ['Age', 'Income', 'Student', 'Credit']
S0 = <0, 0, 0, 0>
G0 = <?, ?, ?, ?>

Example 1: ['Youth', 'Medium', 'Yes', 'Fair'] -> Yes
S = <Youth, Medium, Yes, Fair>
G = <?, ?, ?, ?>

Example 2: ['Youth', 'Low', 'Yes', 'Excellent'] -> Yes
```

S = <Youth, ?, Yes, ?>
G = <?, ?, ?, ?>

Example 3: ['Senior', 'High', 'No', 'Fair'] -> No
S = <Youth, ?, Yes, ?>
G = <Youth, ?, ?, ?> <?, ?, Yes, ?>

Example 4: ['Youth', 'High', 'No', 'Fair'] -> No
S = <Youth, ?, Yes, ?>
G = <?, ?, Yes, ?>

Example 5: ['Middle', 'Medium', 'No', 'Excellent'] -> No
S = <Youth, ?, Yes, ?>
G = <?, ?, Yes, ?>

Final Specific Boundary S: <Youth, ?, Yes, ?>
Final General Boundary G : <?, ?, Yes, ?>

Result

The final boundaries are S = <Youth, ?, Yes, ?> and G = {<?, ?, Yes, ?>}. Here G is more specific than before because the negative example (Youth, High, No, Fair) removed <Youth, ?, ?, ?> from G, so Student = Yes is the key attribute.

Question 4: Loan Approval

Problem Statement

Loan applications are stored in loan.csv. Use Candidate-Elimination to find the version space of hypotheses consistent with approved loans.

Training data (loan.csv):

Income	Job	Credit	Property	Approved
High	Salaried	Good	Own	Yes
High	Salaried	Good	Rent	Yes
Low	Unemployed	Poor	Rent	No
Medium	Business	Poor	Own	No
High	Salaried	Poor	Own	No

Aim

To read loan applicant data from a CSV file and find the set of all hypotheses consistent with approved loans using the Candidate-Elimination algorithm.

Algorithm

1. Read the training examples from the CSV file (last column is the class label).
2. Initialise G to the most general hypothesis {<?, ?, ..., ?>} and S to the most specific hypothesis {<0, 0, ..., 0>}.

3. For each positive example: remove from G every hypothesis that does not cover it.
4. For each hypothesis s in S that does not cover the positive example, replace it by its minimal generalization.
5. For each negative example: remove from S every hypothesis that covers it.
6. For each hypothesis g in G that covers the negative example, replace it by its minimal specializations that are still more general than some member of S, and keep only the most general ones.
7. Print S and G after every example and output the final S and G, which describe the version space.

Python Code

```
import csv

def load(fn):
    with open(fn) as f:
        r = list(csv.reader(f))
    return r[0], r[1:]

def covers(h, x):
    return all(a == "?" or a == b
               for a, b in zip(h, x))

def more_gen(h1, h2):
    return all(a == "?" or a == b or b == "0"
```

```
            for a, b in zip(h1, h2))

def fmt(H):
    return " ".join("<" + ", ".join(h) + ">"
                    for h in H)

def uniq(hs):
    out = []
    for h in hs:
        if h not in out:
            out.append(h)
    return out

def candidate_elimination(rows):
    n = len(rows[0]) - 1
    dom = [sorted({r[i] for r in rows})
           for i in range(n)]
    S, G = [["0"] * n], [["?"] * n]
    print("S0 =", fmt(S))
    print("G0 =", fmt(G))
    for t, r in enumerate(rows, 1):
        x, label = r[:-1], r[-1]
        if label == "Yes":
```

```
            G = [g for g in G if covers(g, x)]
            S = [[xv if sv == "0" else
                  (sv if sv == xv else "?")
                  for sv, xv in zip(s, x)]
                 for s in S]
        else:
            S = [s for s in S if not covers(s, x)]
            new = []
            for g in G:
                if not covers(g, x):
                    new.append(g)
                    continue
                for i in range(n):
                    if g[i] != "?":
                        continue
                    for v in dom[i]:
                        if v != x[i]:
                            h = g[:]
                            h[i] = v
                            if any(more_gen(h, s)
                                   for s in S):
                                new.append(h)
            G = [g for g in uniq(new)
```

```
            if not any(g != k and more_gen(k, g)
                       for k in new)]
        print(f"\nExample {t}: {x} -> {label}")
        print("S =", fmt(S))
        print("G =", fmt(G))
    return S, G

header, rows = load("loan.csv")
print("Attributes:", header[:-1])
S, G = candidate_elimination(rows)
print("\nFinal Specific Boundary S:", fmt(S))
print("Final General Boundary G :", fmt(G))
```

Output

```
Attributes: ['Income', 'Job', 'Credit', 'Property']
S0 = <0, 0, 0, 0>
G0 = <?, ?, ?, ?>

Example 1: ['High', 'Salaried', 'Good', 'Own'] -> Yes
S = <High, Salaried, Good, Own>
G = <?, ?, ?, ?>

Example 2: ['High', 'Salaried', 'Good', 'Rent'] -> Yes
```

S = <High, Salaried, Good, ?>
G = <?, ?, ?, ?>

Example 3: ['Low', 'Unemployed', 'Poor', 'Rent'] -> No
S = <High, Salaried, Good, ?>
G = <High, ?, ?, ?> <?, Salaried, ?, ?> <?, ?, Good, ?>

Example 4: ['Medium', 'Business', 'Poor', 'Own'] -> No
S = <High, Salaried, Good, ?>
G = <High, ?, ?, ?> <?, Salaried, ?, ?> <?, ?, Good, ?>

Example 5: ['High', 'Salaried', 'Poor', 'Own'] -> No
S = <High, Salaried, Good, ?>
G = <?, ?, Good, ?>

Final Specific Boundary S: <High, Salaried, Good, ?>
Final General Boundary G : <?, ?, Good, ?>

Result

The final boundaries are S = <High, Salaried, Good, ?> and G = {<?, ?, Good, ?>}. Good credit is the single most general condition for approval, and the most specific one is High income, Salaried job and Good credit.

Question 5: Medical Diagnosis (Flu) using a Class

Problem Statement

Patient records are stored in flu.csv. Implement Candidate-Elimination as a Python class, find the version space for Flu and classify new patients with it.

Training data (flu.csv):

Fever	Cough	BodyTemp	Aches	Flu
Yes	Yes	High	Yes	Yes
Yes	No	High	Yes	Yes
No	No	Normal	No	No
Yes	Yes	Normal	No	No

Aim

To build an object-oriented Candidate-Elimination learner that reads a CSV file, outputs the S and G boundaries for Flu and classifies new patients.

Algorithm

1. Read the training examples from the CSV file (last column is the class label).
2. Initialise G to the most general hypothesis {<?, ?, ..., ?>} and S to the most specific hypothesis {<0, 0, ..., 0>}.
3. For each positive example: remove from G every hypothesis that does not cover it.

4. For each hypothesis s in S that does not cover the positive example, replace it by its minimal generalization.
5. For each negative example: remove from S every hypothesis that covers it.
6. For each hypothesis g in G that covers the negative example, replace it by its minimal specializations that are still more general than some member of S, and keep only the most general ones.
7. Print S and G after every example and output the final S and G, which describe the version space.

Python Code

```
import csv

class CandidateElimination:
    def __init__(self, n, domains):
        self.n, self.dom = n, domains
        self.S = [["0"] * n]
        self.G = [["?"] * n]

    @staticmethod
    def covers(h, x):
        return all(a == "?" or a == b
                   for a, b in zip(h, x))

    @staticmethod
```

```
def more_gen(h1, h2):
    return all(a == "?" or a == b or b == "0"
               for a, b in zip(h1, h2))

def update(self, x, positive):
    if positive:
        self.G = [g for g in self.G
                  if self.covers(g, x)]
        self.S = [[xv if sv == "0" else
                   (sv if sv == xv else "?")
                   for sv, xv in zip(s, x)]
                  for s in self.S]
        return
    self.S = [s for s in self.S
              if not self.covers(s, x)]
    new = []
    for g in self.G:
        if not self.covers(g, x):
            new.append(g)
            continue
        for i in range(self.n):
            if g[i] != "?":
                continue
```

```
            for v in self.dom[i]:
                if v == x[i]:
                    continue
                h = g[:]
                h[i] = v
                if any(self.more_gen(h, s)
                       for s in self.S):
                    new.append(h)
        keep = []
        for g in new:
            top = not any(g != k and
                          self.more_gen(k, g)
                          for k in new)
            if top and g not in keep:
                keep.append(g)
        self.G = keep

    def classify(self, x):
        if all(self.covers(s, x) for s in self.S):
            return "Flu"
        if not any(self.covers(g, x)
                   for g in self.G):
            return "No Flu"
```

```
        return "Uncertain"

def fmt(H):
    return " ".join("<" + ", ".join(h) + ">"
                    for h in H)

with open("flu.csv") as f:
    rows = list(csv.DictReader(f))
cols = list(rows[0].keys())
feat, target = cols[:-1], cols[-1]
dom = [sorted({r[c] for r in rows})
       for c in feat]

ce = CandidateElimination(len(feat), dom)
print("Features:", feat)
for t, r in enumerate(rows, 1):
    x = [r[c] for c in feat]
    ce.update(x, r[target] == "Yes")
    print(f"\nExample {t}: {x} -> {r[target]}")
    print("S =", fmt(ce.S))
    print("G =", fmt(ce.G))

print("\nFinal S:", fmt(ce.S))
```

```
print("Final G:", fmt(ce.G))
print("\nTest 1:", ce.classify(
    ["Yes", "No", "High", "Yes"]))
print("Test 2:", ce.classify(
    ["No", "No", "Normal", "No"]))
print("Test 3:", ce.classify(
    ["Yes", "Yes", "High", "No"]))
```

Output

```
Features: ['Fever', 'Cough', 'BodyTemp', 'Aches']

Example 1: ['Yes', 'Yes', 'High', 'Yes'] -> Yes
S = <Yes, Yes, High, Yes>
G = <?, ?, ?, ?>

Example 2: ['Yes', 'No', 'High', 'Yes'] -> Yes
S = <Yes, ?, High, Yes>
G = <?, ?, ?, ?>

Example 3: ['No', 'No', 'Normal', 'No'] -> No
S = <Yes, ?, High, Yes>
G = <Yes, ?, ?, ?> <?, ?, High, ?> <?, ?, ?, Yes>
```

Example 4: ['Yes', 'Yes', 'Normal', 'No'] -> No
S = <Yes, ?, High, Yes>
G = <?, ?, High, ?> <?, ?, ?, Yes>

Final S: <Yes, ?, High, Yes>
Final G: <?, ?, High, ?> <?, ?, ?, Yes>

Test 1: Flu
Test 2: No Flu
Test 3: Uncertain

Result

The final boundaries are S = <Yes, ?, High, Yes> and G = {<?, ?, High, ?>, <?, ?, ?, Yes>}. Test 1 was classified as Flu and Test 2 as No Flu; Test 3 is Uncertain because the hypotheses in the version space disagree.